Zestaw nr 3

2025.10.23, Marcin Zagórski

Zadanie 1 (4 pkt). Granice ciągów, Limit, DiscreteLimit, Plot, DiscretePlot

Policz granice następujących ciągów:

1) $\lim_{n\to\infty}\left(1+\frac{a}{n}\right)^n$
2) $\lim_{n\to\infty} n\sin\frac{1}{n}$
3) $\lim_{n\to\infty} n\ln\left(1+\frac{1}{n}\right)$
4) $\lim_{n\to\infty} \sin(n)\ \ln\left(1+\frac{1}{n}\right)$
5) $\lim_{n\to\infty} \frac{e^n}{n}$
6) $\lim_{n\to\infty} (-1)^n \frac{e^n}{n}$
7) $\lim_{n\to\infty} \sin(n)\frac{e^n}{n}$
8) $\lim_{n\to\infty} \sin(n\,\pi)\frac{e^n}{n}$
9) $\lim_{n\to\infty}\left(\left(\sum_{k=1}^{n}\frac{1}{k}\right)-\ln n\right)$

> Wskazówka: By szybko wpisywać wyrażenia matematyczne z potęgami korzystaj ze skrótu Ctrl+^, podobnie dla wpisywania ułamków użyteczny jest skrót Ctrl+/ .

Jak należy interpretować wyniki ComplexInfinity, Indeterminate? Czy w powyższych przykładach jest różnica między użyciem funkcji Limit, a funkcji DiscreteLimit?

Narysuj ciąg (funkcję) $f(n)=\sin(n\,\pi)\frac{e^n}{n}$ korzystając z funkcji Plot w zakresie $1\le n\le 30$. Następnie narysuj ten sam ciąg korzystając z DiscretePlot.

> Ciekawostka: Szukana granica w 9) to tzw. stała Eulera-Mascheroniego, γ, do tej pory nie wiadomo, czy stała γ jest liczbą wymierną, czy niewymierną.

Zadanie 2 (3 pkt). Szeregi liczbowe, Sum

Rozważ płaski ekran ustawiony w punkcie $x=0$, na który pada światło od ciągu n punktowych źródeł światła umieszczonych na osi x w odległościach $x_1=1\ \mathrm{m}, x_2=2\ \mathrm{m}, \dots, x_n=n\ \mathrm{m}$ od ekranu. Każde ze świateł świeci z taką samą mocą, a natężenie światła padającego na ekran od źródła w odległości x_1 wynosi $1\ \mathrm{lx}$ ($1\ \mathrm{luks}=\frac{1\ \mathrm{lumen}}{\mathrm{m}^2}$). Jakie będzie całkowite natężenie światła padającego na ekran dla $n\to\infty$?

> Ciekawostka: Sumę szeregu, który jest rozwiązaniem tego zadania można wyznaczyć również geometrycznie, https://www.youtube.com/watch?v=d-o3eB9sfls (17 min)

Zadanie 3 (4 pkt). Szeregi liczbowe, Sum, ListPlot, ListLogPlot

Policz granicę następującego ciągu $S_k=\sum_{n=1}^{\infty}\frac{1}{n^k}$, którego elementami są sumy szeregów. Najpierw sprawdź jak zmienia się S_k dla k w zakresie $2\le k\le k_{max}$. Wynik zapisz w postaci listy $sk=\left\{\{2,S_2\},\{3,S_3\},\dots,\{k_{max},S_{k_{max}}\}\right\}$. Następnie:

1) Wypisz sk dla $k_{max}=20$. Jakie wartości przyjmują elementy tej listy dla parzystych k?
2) Przedstaw na wykresie korzystając z funkcji ListPlot wartości sk dla $k_{max}=100$.
3) Utwórz listę $sk1=\left\{\{2,S_2-1\},\{3,S_3-1\},\dots,\{k_{max},S_{k_{max}}-1\}\right\}$ dla $k_{max}=100$.
4) Przedstaw listę $sk1$ na wykresie ListLogPlot. Co sugeruje ten wykres? Jaka jest granica ciągu S_k dla $k\to\infty$?

Wskazówka: Jak zachowuje się wartość $S_k - 1$ przy zwiększaniu k. Dlaczego by utworzyć $sk1$ odjęliśmy wartość 1? Sprawdź wynik działania $\mathrm{Limit}[\mathrm{Zeta}[n], n \to \infty]$.

Zadanie 4 (3 pkt). Szeregi liczbowe, SumConvergence, Sum, FullSimplify

Sprawdź czy szereg

$$S_n = \sum_{k=0}^{n} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right) 16^{-k}$$

jest zbieżny dla $n \to \infty$? Użyj funkcji $\mathrm{SumConvergence}$.

Policz granicę S_n korzystając z Sum. Wypisz tę wartość numerycznie. Wynik działania funkcji Sum uprość przy pomocy $\mathrm{FullSimplify}$.

Ciekawostka: Jest to tak zwany szereg Bailey-Borwein-Plouffe, został odkryty w 1995 przy pomocy algorytmu PSQL, zaimplementowany w bazie szesnastkowej pozwala obliczać dalekie cyfry rozwinięcia liczby π bez obliczania poprzedzających cyfr. Więcej informacji:
https://observablehq.com/@rreusser/computing-with-the-bailey-borwein-plouffe-formula

Zadanie 5 (6 pkt). Błądzenie przypadkowe w 1D, Module, RandomChoice, Accumulate, Series oraz inne

Zdefiniuj proces, w którym kulka porusza się po osi liczb całkowitych, wykonując skoki w lewo ($\Delta x = -1$) albo w prawo ($\Delta x = +1$) z takim samym prawdopodobieństwem $p_L = p_R = 0.5$. Jeśli kulka zaczyna błądzenie w punkcie $x = 0$, to jakie jest prawdopodobieństwo p_n, że po dokładnie n skokach znowu znajdzie się w położeniu początkowym? Dla ustalenia uwagi, rozwiąż dla $n = 1000$ skoków.

Zacznij od zdefiniowania własnej funkcji losującej serię n skoków:

$$\mathrm{getSteps}[n_] := \mathrm{RandomChoice}[\{-1,1\}, n]$$

gdzie -1 odpowiada skokowi w lewo, a $+1$ skokowi w prawo.

Następnie zdefiniuj funkcję zwracającą pozycję kulki po $1, 2, \ldots n$ skokach:

$$\begin{aligned}&\mathrm{getWalk}[\mathrm{n_}] := \mathrm{Module}[\{\mathrm{steps}, \mathrm{walk}\},\\ &\quad \mathrm{steps} = \mathrm{getSteps}[\mathrm{n}];\\ &\quad \mathrm{walk} = \mathrm{Accumulate}[\mathrm{steps}];\\ &\quad \mathrm{walk}]\end{aligned}$$

Sprawdź działanie funkcji $\mathrm{getWalk}$ przedstawiając jej wynik na wykresie. Następnie:

1) Oszacuj prawdopodobieństwo p_{1000} numerycznie. Przykładowo wygeneruj $n = 1000$ niezależnych błądzeń losowych, sprawdź w ilu z tych błądzeń, $q_n^{x=0}$, kulka kończyła błądzenie w położeniu $x = 0$. Oszacowanie na p_{1000} to $q_{1000}^{x=0}/1000$.
2) Potraktuj punkt 1) jako pojedynczy eksperyment. Wykonaj teraz ten eksperyment $n_R = 100$ razy. Zapisz wyniki, policz ich średnią, odchylenie standardowe, oraz odchylenie standardowe średniej. Jak zmienią się wyniki, gdy zwiększysz liczbę powtórzeń do $n_R = 400$.
3) Wartość p_n można również obliczyć bezpośrednio (analitycznie). Zastanów się ile razy musi wystąpić $-1/+1$ w trakcie błądzenia przypadkowego o n skokach by kulka wróciła do położenia początkowego. Na ile sposobów można tak wylosować ciąg -1 oraz $+1$ o długości n? Ile jest wszystkich możliwych kombinacji -1 oraz $+1$ w ciągu o długości n?

Wynik możesz przekształcić korzystając ze wzoru Stirlinga na silnię, albo skorzystaj z polecenia Series by rozwinąć zależność od n względem ∞, wystarczy do 1 stopnia (wyraz wiodący).

4) Czy wynik z punktu 2) zgadza się z wynikiem teoretycznym z punktu 3) w granicach błędu?

Komentarz: Funkcja getWalk robi więcej niż wymaga zadanie, ale pozwala na graficzne przedstawienie błądzenia kulki. Jeśli zależy nam na szybkości wykonywanych obliczeń, możemy zdefiniować funkcję getWalkLast podobnie do getWalk przez zastąpienie w definicji funkcji Accumulate przez Total. O ile szybciej wykonają się powyższe obliczenia?